

University of Bologna
Master's student in Artificial Intelligence

Transformer inference time cost analysis for protein fitness prediction

Full-Sequence vs RMT-Style Block-Recurrent Inference

A Theoretical and Empirical Cost–Accuracy Analysis
on **ProteinGym** DMS Substitution Assays

Author: Faezeh Sarlakifar
Course: APAI (Architecture and Platforms for Artificial Intelligence)
Credits: 3 CFU
Supervisors: Prof. Luca Benini, Prof. Francesco Conti
Tutors: Andrea Belano, Alberto Dequino, Calin Diaconu
Academic Year: 2025-2026

August 10, 2026

Abstract

This report presents a theoretical and empirical cost analysis comparing two inference-time strategies for protein language models: standard full-sequence self-attention and a Recurrent Memory Transformer (RMT)-style block-recurrent scheme built on top of a frozen, pretrained ESM-2 backbone. The theoretical section derives instruction-level asymptotic time and memory complexity for both approaches, establishing $O(n^2)$ time and $O(n^2)$ peak activation memory for full-sequence attention against $O(n)$ time and $O(b^2)$ (sequence-length-independent) peak memory for the RMT variant, at the cost of forced sequential processing across segments and a fixed-size memory bottleneck. The empirical section measures actual MACs (via `fvcore`) and peak GPU memory (via `torch.cuda`) on real ProteinGym Deep Mutational Scanning (DMS) sequences across two ESM-2 checkpoints (35M and 150M parameters) and four block sizes ($b \in \{128, 64, 32, 16\}$), and evaluates zero-shot fitness-prediction accuracy via masked-marginal scoring correlated (Spearman ρ) against experimental DMS scores. A targeted ablation isolates the individual contributions of memory-token count and of the recurrence mechanism itself. The central empirical finding is that on the longest available sequence (3423 residues), the full-sequence 150M model exhausts GPU memory entirely, while the RMT variant completes the same forward pass using less memory than the full-sequence baseline uses on far shorter proteins – confirming the theoretical memory-scaling advantage in a real out-of-memory boundary rather than only in an asymptotic curve. This memory advantage is shown to come at a real, measured accuracy cost that degrades monotonically as block size shrinks.

1 Introduction and Motivation

Protein language models built on the Transformer architecture, such as ESM-2, use full self-attention over the entire input sequence. This gives every residue direct access to every other residue in a single forward pass, which is part of why these models are effective at zero-shot fitness prediction. It is also the source of their central scalability problem: self-attention’s time and memory cost both grow quadratically with sequence length, $O(n^2)$, so long proteins are disproportionately expensive to score, and beyond some length the model simply does not fit in GPU memory at all.

This project investigates whether that quadratic cost can be converted into a linear one by retrofitting a pretrained ESM-2 checkpoint with a Recurrent Memory Transformer (RMT)-style inference procedure, without retraining the underlying model. The core idea is to split the input sequence into fixed-size blocks and carry a small set of learned memory tokens across blocks, so that only one block’s worth of tokens is ever attended over at a time. This report is organized in two parts, matching the two stages of the investigation:

1. A **theoretical, instruction-by-instruction cost analysis** of both inference procedures, deriving their asymptotic time and memory complexity in terms of sequence length n (Section 2).
2. An **empirical cost and accuracy analysis**, measuring actual MACs, peak GPU memory, and zero-shot fitness-prediction accuracy on real ProteinGym sequences, across multiple model sizes and block sizes, plus a targeted ablation isolating which structural components of the RMT mechanism are actually responsible for its behavior (Sections 3–4).

The dataset used throughout is ProteinGym’s Deep Mutational Scanning (DMS) substitution benchmark, publicly available at <https://proteingym.org/>. Each DMS assay consists of a single wildtype protein along with a large panel of experimentally measured single- and multi-mutant variants; because substitution mutations do not insert or delete residues, every variant within one assay shares exactly the wildtype’s sequence length.

2 Theoretical Cost Analysis

2.1 Which algorithm, and why

The inference-time algorithm evaluated in this project is an **RMT-style segment-recurrent inference procedure**: the transformer backbone (ESM-2) is left entirely untouched, but the input sequence is split into fixed-size blocks, and a small set of learned “memory tokens” is carried across blocks to pass information forward. This is compared throughout against the **full-sequence baseline** – plain self-attention over the whole protein in a single pass.

The choice of RMT over the alternatives considered during the literature review is motivated as follows:

- **vs. full-sequence attention**: full attention is exact but scales quadratically in sequence length – this is precisely the bottleneck the project targets.
- **vs. Block-Recurrent Transformer**: more expressive (an LSTM-gated recurrent cell inside a full transformer layer), but it requires a custom recurrent cell trained from scratch. This is out of scope for a

three-week, 3-CFU project with no budget for training a new architecture, only for retrofitting an existing one.

- **vs. Universal Transformer:** recurrence over depth (weight-tying across layers), not over sequence position. It addresses parameter count rather than the sequence-length cost this project targets, so it is not a fair substitute.
- **vs. sparse/linear attention (Longformer, Performer, sliding-window):** legitimate alternatives, but most require either a custom attention kernel or fine-tuning to adapt pretrained weights to a new attention pattern. RMT’s appeal is that it is a bolt-on: the pretrained ESM-2 weights are used as-is, with memory-token machinery added around them.

Pros of RMT. Retrofittable onto a pretrained model (fits the project timeframe); memory footprint independent of total sequence length; straightforward to implement and monitor.

Cons of RMT. It is an approximation: long-range dependencies must be compressed through a fixed-size memory bottleneck, so some accuracy loss relative to exact full attention is expected. Segments must be processed in order – the memory state at block t depends on block $t - 1$ – so RMT is not embarrassingly parallel across the sequence the way full attention is. In addition, the block-splitting strategy could plausibly be informed by biological structure (e.g. motifs, domains) rather than a fixed arbitrary size, which is discussed further in Section 5.1.

2.2 Setup and notation

- n = protein sequence length (the variable this analysis is measured against)
- d = hidden dimension
- L = number of transformer layers
- b = block size
- m = number of memory tokens ($m \ll b$)
- $s = \lceil n/b \rceil$ = number of segments

d , b , and L are fixed model and hyperparameter choices, not functions of input length, and are therefore treated as constants throughout. This holds only as long as b itself is capped independently of n ; if block size were instead scaled with sequence length (e.g. $b = n/4$), RMT’s linear-time property would disappear.

2.3 Instruction-by-instruction cost breakdown

Listing 1: RMT-style segment-recurrent inference procedure

```

Algorithm RMT_Inference(X, M, b, m)

1. Split X into segments X_1 ... X_s, s = ceil(n / b)
2. Mem_0 <- init memory state (size m x d)
3. for t = 1 .. s:
3a. Z_t <- concat(Mem_{t-1}, X_t) # length b + m
3b. for l = 1 .. L:
        Q, K, V <- Z_t W_Q, Z_t W_K, Z_t W_V
        A <- softmax(Q K^T / sqrt(d))
        O <- A V
        Z_t <- FFN(OutProj(O) + Z_t) # + residual
3c. Mem_t, Y_t <- split(Z_t) # recover updated memory + segment output
3d. store Y_t; carry Mem_t forward
4. Y <- concat(Y_1 ... Y_s)

```

The outer loop (step 3) runs $s = n/b$ times, giving total cost

$$T_{\text{RMT}}(n) = O\left(\frac{n}{b} \cdot L \cdot ((b+m)^2 d + (b+m)d^2)\right).$$

Since $m \ll b$, $(b+m) \approx b$, which simplifies to

$$T_{\text{RMT}}(n) = O(L \cdot n \cdot b \cdot d + L \cdot n \cdot d^2) = O(n) \quad \text{for fixed } b, d, L.$$

Table 1: Per-instruction cost of one RMT segment step

Step / Operation	Cost
Split sequence into segments	$O(n)$
Init memory	$O(md)$
Concat memory + segment (3a)	$O((b+m)d)$
QKV projections	$O((b+m)d^2)$
$QK^\top$	$O((b+m)^2d)$
Softmax	$O((b+m)^2)$
$A \cdot V$	$O((b+m)^2d)$
Output projection + FFN	$O((b+m)d^2)$
Per layer, summed	$O((b+m)^2d + (b+m)d^2)$
Per segment, $\times L$ layers	$O(L \cdot ((b+m)^2d + (b+m)d^2))$
Split + store (3c/3d)	$O((b+m)d)$
Concat outputs (step 4)	$O(n)$

Table 2: Theoretical comparison: full-sequence attention vs. RMT block-recurrence

	Full-sequence attention	RMT block-recurrent
Time (total FLOPs)	$O(L(n^2d + nd^2))$	$O(L(nbd + nd^2)) \rightarrow O(n)$, fixed b
Peak activation memory	$O(n^2)$ (attention matrix over whole sequence)	$O(b^2)$ (attention matrix per block) $\rightarrow$ independent of n
Parallelism / latency	one forward pass, fully parallel across n	s sequential segment passes (memory dependency chain) – not parallel across the whole sequence
Accuracy	exact global context	lossy: long-range information compressed through an m -token bottleneck

2.4 Comparison against the full-sequence baseline

For full attention, $s = 1$ and the effective block size equals n , giving

$$T_{\text{full}}(n) = O(L \cdot (n^2d + nd^2)) = O(n^2) \quad \text{for fixed } d, L.$$

This is the trade-off the empirical measurements in Section 4 are designed to probe directly: RMT converts the quadratic-in- n attention cost into a linear one and caps peak memory at a constant independent of protein length, at the price of (a) forced sequential processing across segments and (b) a compression bottleneck that can hurt fitness-prediction accuracy, particularly on long proteins.

2.5 From ideal to effective: what the theory does not capture

The instruction count above describes an *ideal* cost model: it assumes every operation costs exactly what its arithmetic shape implies, with no constant-factor overhead. Real measurements diverge from this ideal in ways worth stating explicitly before presenting them:

- **Fixed-percentage tax.** Every RMT segment carries m memory tokens in addition to its b real tokens, so RMT does strictly more arithmetic than a hypothetical linear-only baseline – specifically, an overhead close to m/b relative to a same-length non-recurrent computation, independent of n . This shows up as a roughly constant percentage gap between RMT and full-sequence MACs at any given n , not something that shrinks as n grows.
- **Latency vs. FLOPs.** The theoretical time complexity counts total arithmetic (FLOPs / MACs), not wall-clock latency. RMT’s s sequential segment passes mean real latency includes s sequential kernel launches and synchronization points that a pure FLOP count does not capture – a hardware-level cost the asymptotic analysis intentionally abstracts away.
- **Which term dominates at a given n .** The quadratic attention term in $T_{\text{full}}(n)$ only dominates the linear projection term $O(nd^2)$ once n becomes large relative to d . At moderate sequence lengths, both full-sequence and RMT can appear to scale similarly (both close to linear), and the crossover into visibly quadratic behavior for the baseline may not appear within the range of naturally occurring protein lengths

– this is examined directly against the empirical MACs data in Section 4.1.

3 Experimental Methodology

3.1 Models

Two ESM-2 checkpoints from Hugging Face were used throughout, chosen to span roughly a $4\times$ parameter range while remaining small enough to profile and iterate on within the project timeframe:

- `esm2_t12_35M_UR50D` (35M parameters)
- `esm2_t30_150M_UR50D` (150M parameters)

For cost measurement, models are loaded via `AutoModel` (bare encoder, no language-modeling head). For accuracy evaluation, the same checkpoints are loaded via `AutoModelForMaskedLM`, since zero-shot scoring requires vocabulary logits.

3.2 Dataset

Sequences are drawn from ProteinGym’s DMS substitution benchmark (`OATML-Markslab/ProteinGym_v1`, config `DMS_substitutions`). A stratified subset of six assays was selected, two from each of three length buckets (short: < 200 aa; medium: $200\text{--}500$ aa; long: > 500 aa), listed in Table 3. This is the same subset used for both the cost sweep and the accuracy comparison, so both measurements are computed on identical real proteins rather than independently sampled pools.

Table 3: Stratified assay subset used throughout this analysis

DMS assay	Bucket	Length (residues)
SQSTM.MOUSE.Tsuboyama_2023_2RRU	short	40
SUMO1.HUMAN.Weile_2017	short	101
TRPC.SACS2.Chan_2017	medium	248
SPG1.STRSG.Wu_2016	medium	448
A4.HUMAN.Seuma_2022	long	770
A0A140D2T1.ZIKV.Sourisseau_2019	long	3423

Critically, every cost-sweep data point uses a real, whole ProteinGym sequence at its own natural length – not a random-integer dummy input, and not a sequence artificially truncated or tiled to hit a round target length. This was a deliberate late-stage change from an earlier version of the pipeline that matched real sequences to a fixed target-length list with a tolerance window; the final version instead sweeps directly over the lengths actually present in the assay subset above.

3.3 Cost measurement

For each (model, variant, assay) combination, MACs are measured via `fvcore.nn.FlopCountAnalysis` and peak GPU memory via `torch.cuda.max_memory_allocated()` (reset between runs), averaged over three repeated measurements. Two variants are compared for each block size $b \in \{128, 64, 32, 16\}$: the full-sequence baseline (identical across all block-size sweeps, since it does not depend on b) and the RMT wrapper with $m = 8$ memory tokens. One (model, block size) combination – `esm2_t30_150M_UR50D` at $b = 16$ – was found to crash the runtime outright during RMT evaluation rather than raising a catchable CUDA out-of-memory exception, and is therefore recorded as OOM by construction without being executed, to keep the sweep from taking down the whole session.

3.4 Accuracy measurement

Zero-shot fitness-prediction accuracy is measured via masked-marginal scoring, matching the method established in the project’s Week 1 baseline: for each mutated position, the wildtype sequence is masked at that position and scored in a single forward pass; the predicted effect of a mutation is $\log P(\text{mutant AA}) - \log P(\text{wildtype AA})$ at that position. Since ProteinGym’s DMS rows store only the mutant sequence, the wildtype is reconstructed for single mutants by reverting the one substitution encoded in the `mutant` field. Predicted scores are correlated (Spearman ρ) against each assay’s experimental `DMS_score`, for every (model, variant) pair across all six assays, with block sizes $b \in \{128, 64, 32, 16\}$.

3.5 Ablation design

Two structural components of the RMT mechanism are isolated in a dedicated ablation, since sweeping block size alone does not separate them:

- **Memory-token count** $m \in \{0, 2, 4, 8, 16\}$. $m = 0$ disables the memory mechanism entirely: the sequence is chopped into independent blocks with zero cross-block information flow.
- **carry_memory (recurrence)**: when true, the memory state passed into block $t + 1$ is block t 's actual updated output (real recurrence, matching the algorithm in Listing 1); when false, memory resets to its untrained initial value at the start of every block, regardless of what came before.

The ablation was run on `esm2_t12_35M.UR50D` with $b = 32$, cost measured on the single longest available sequence (`A0A140D2T1_ZIKV_Sourisseau.2019`, $n = 3423$), and accuracy averaged over four of the six assays for tractability.

4 Results

4.1 Cost sweep across sequence length

Figure 1 shows MACs and peak memory as a function of sequence length, for both models, at block size $b = 128$. The headline result of this entire analysis is visible directly in the raw numbers behind this figure, summarized in Table 4: on the longest available sequence ($n = 3423$), the `esm2_t30_150M.UR50D` full-sequence baseline exhausts GPU memory entirely, while the RMT variant of the same checkpoint completes the forward pass using **less** peak memory (724.2 MB) than the full-sequence baseline uses on proteins an order of magnitude shorter (744.3 MB at $n = 770$).

Table 4: The central empirical result: full-sequence OOM vs. RMT survival at $n = 3423$ (`esm2_t30_150M.UR50D`)

Variant	Seq. length	MACs	Peak memory (MB)
full-sequence	770	1.137×10^{11}	744.25
full-sequence	3423	OOM	OOM
RMT ($b = 128$)	3423	5.373×10^{11}	724.20

This is not a marginal, asymptotic advantage that only shows up at extreme scale – it is a real, hard capability boundary the full-sequence baseline crosses on a naturally occurring protein length in this dataset, and RMT does not.

At shorter lengths, RMT consistently costs *more* MACs than the full-sequence baseline (visible as the RMT curve sitting slightly above full-sequence in the left panel of Figure 1 at every n), consistent with the fixed-percentage tax identified in Section 2: RMT always processes $m = 8$ extra memory-token positions per segment on top of the real tokens, so it does strictly more arithmetic whenever both approaches fit in memory. The two lines in the MACs panel remain roughly parallel across the full range tested (up to $n = 3423$), rather than visibly diverging – indicating that, in agreement with the theoretical discussion in Section 2, the linear projection term $O(nd^2)$ still dominates the quadratic attention term $O(n^2d)$ at these lengths for both model sizes, and the crossover into clearly quadratic full-sequence cost growth is not yet visible in MACs terms within the range of naturally occurring protein lengths sampled here. The advantage that *is* clearly visible at this scale is the memory one, not the compute one.

4.2 Effect of block size on cost

Figure 5 and Figure 6 compare all four tested block sizes directly, per model, using per-token MACs (which normalizes out the dominant linear term so differences between block sizes are visible rather than buried in overall magnitude).

Two consistent patterns emerge across both models:

1. **Smaller block sizes cost more MACs per token.** As b shrinks from 128 down to 16, per-token MACs rise. This matches the theoretical fixed-tax argument directly: the relative overhead from the $m = 8$ memory tokens scales as m/b , so halving b roughly doubles the relative tax.
2. **Smaller block sizes save more memory, but with a floor.** Peak memory drops as b shrinks, most sharply once n is large enough that the full-sequence baseline’s quadratic memory term begins to dominate (visible as the steep late rise of the purple full-sequence line in Figure 5, right panel). RMT’s memory stays close to flat across the entire sequence-length range regardless of block size, confirming the $O(b^2)$, sequence-length-independent memory bound from Section 2.

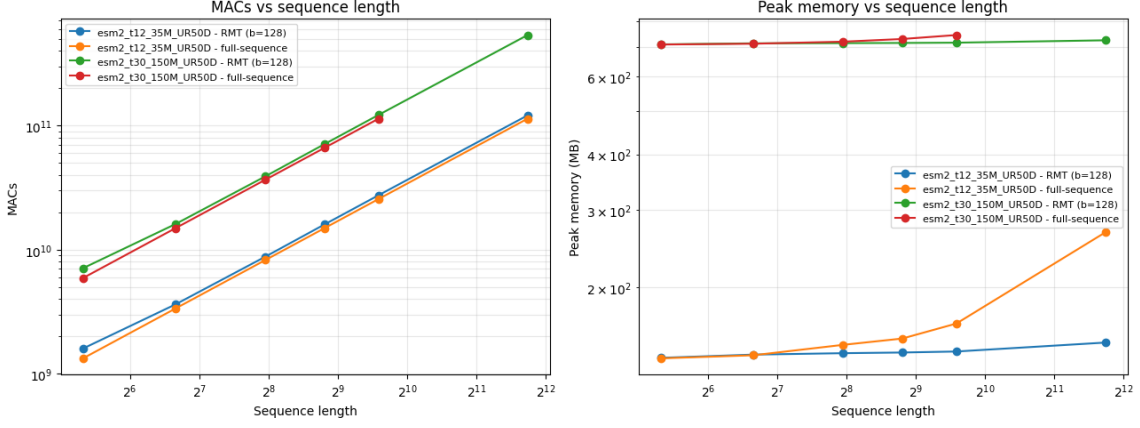


Figure 1: MACs (left) and peak GPU memory (right) vs. sequence length, both models, $b = 128$. Note the missing `esm2.t30.150M.UR50D` full-sequence point at $n = 3423$ (OOM) in the memory panel.

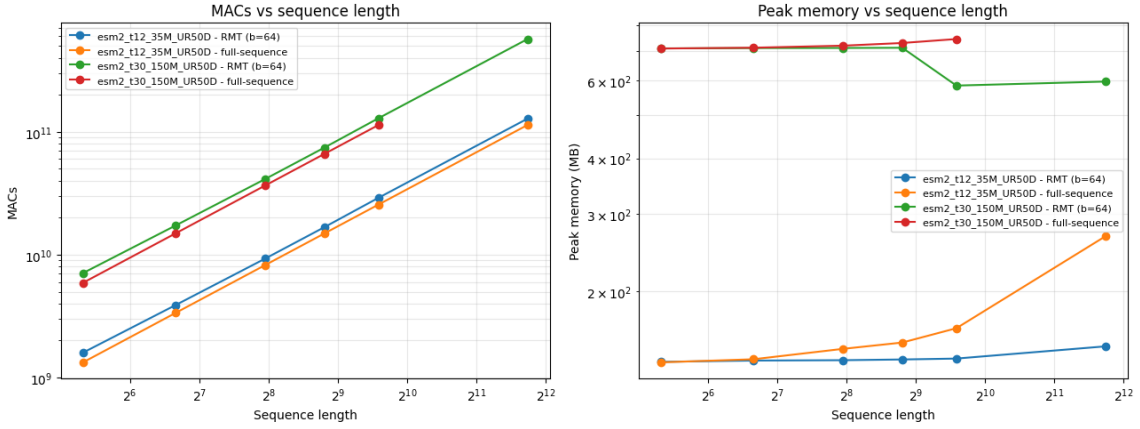


Figure 2: MACs (left) and peak GPU memory (right) vs. sequence length, both models, $b = 64$.

For `esm2.t30.150M.UR50D`, the $b = 16$ curve is entirely absent from Figure 6: this is the runtime-crashing combination described in Section 3, which was excluded from execution by design rather than measured and found equal to zero.

4.3 Accuracy comparison

Figure 7 shows mean Spearman ρ (zero-shot fitness prediction accuracy, averaged across all six assays) for each variant, for both models. The pattern is a clean, monotonic degradation as block size shrinks: full-sequence attention is the most accurate variant for both models, and accuracy falls steadily through $b = 128 \rightarrow 64 \rightarrow 32 \rightarrow 16$.

Table 5 breaks this down by length bucket rather than averaging across all assays. Two details are worth drawing out specifically:

- At $b = 128$, RMT’s accuracy on the *short*-protein bucket is essentially indistinguishable from the full-sequence baseline for both models (0.478 vs. 0.469 for `esm2.t12.35M.UR50D`; 0.530 vs. 0.531 for `esm2.t30.150M.UR50D`). This is expected: when the block size is close to or larger than the sequence itself, RMT degenerates toward a single-segment computation very similar to full attention, so little accuracy should be lost – and the measurement confirms this directly rather than only by construction.
- The *long*-bucket accuracy is low for every variant, including the full-sequence baseline (0.156–0.168 range) – markedly lower than the short and medium buckets. With only two assays per bucket, this bucket average is dominated by a single very difficult assay (`A0A140D2T1.ZIKV.Sourisseau_2019`, whose Spearman ρ is slightly negative for every variant tested), and should not be read as a general finding about long proteins specifically; it is flagged here as a sample-size limitation rather than a conclusion (see Section 6).

4.4 Ablation: memory-token count and recurrence

Three findings follow directly from the measured data:

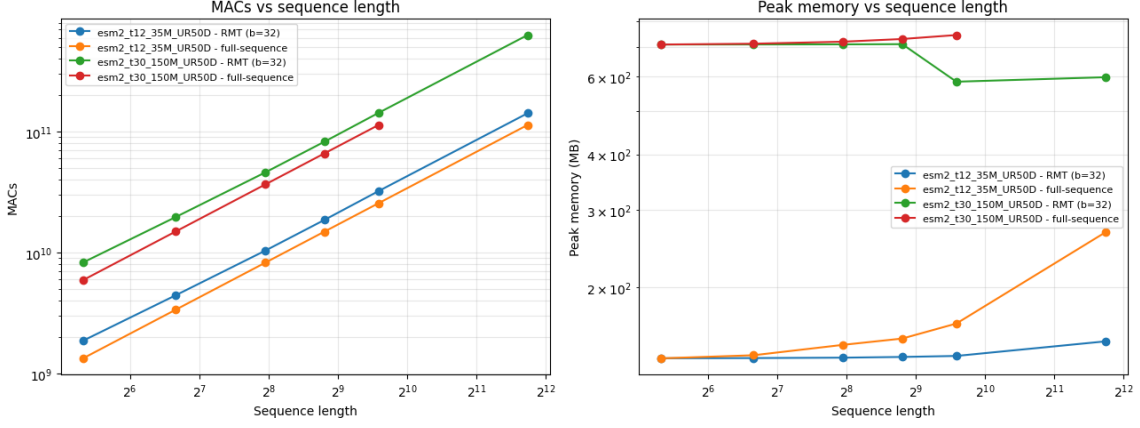


Figure 3: MACs (left) and peak GPU memory (right) vs. sequence length, both models, $b = 32$.

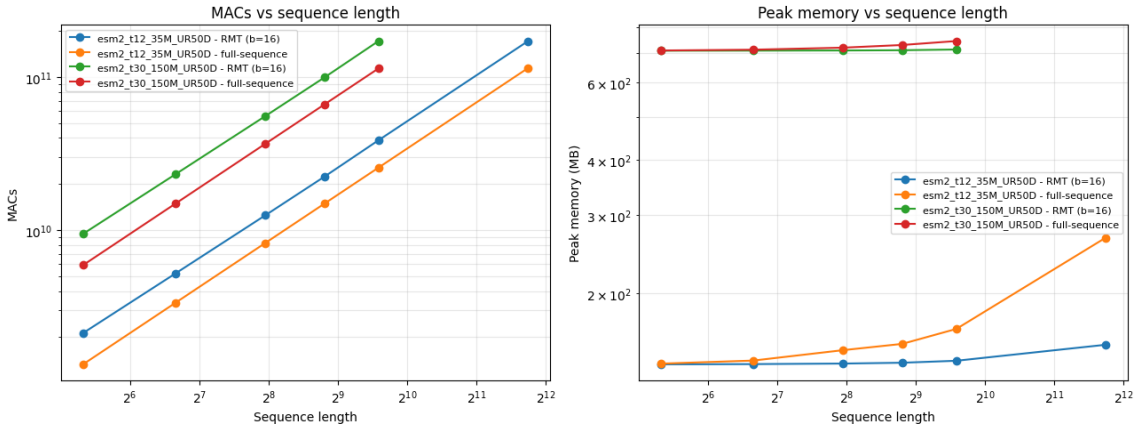


Figure 4: MACs (left) and peak GPU memory (right) vs. sequence length, both models, $b = 16$.

1. **Recurrence is doing real work, not sitting idle.** At every tested $m > 0$, `carry_memory=True` outperforms `carry_memory=False` on accuracy (e.g. at $m = 16$: $\rho = 0.1056$ with carry vs. $\rho = 0.0949$ without). This directly answers the question the ablation was designed to ask: the benefit RMT gets from memory tokens is not just from having *some* local extra tokens for the attention mechanism to use, but specifically from carrying information forward across blocks.
2. **More memory tokens monotonically improve accuracy, at a monotonically increasing MACs cost.** Accuracy rises steadily from $m = 0$ ($\rho = 0.0890$) to $m = 16$ ($\rho = 0.1056$) under real recurrence, while MACs rise from 114.6B to 171.5B over the same range – a genuine, consistent trade-off curve with no free improvement.
3. **Recurrence itself is free in memory terms.** The peak-memory curves for `carry_memory=True` and `carry_memory=False` are numerically identical at every m . Carrying the memory tensor forward instead of resetting it costs nothing extra in peak activation memory – the entire measured benefit of recurrence is upside, with no memory penalty attached.

5 Discussion

Taken together, the theoretical and empirical results tell a consistent story, but with one important nuance the theory alone does not surface. The theoretical analysis in Section 2 frames RMT’s advantage symmetrically across two axes: linear time and constant memory, both against quadratic baselines. The empirical results show these two axes behaving very differently within the range of protein lengths actually available in ProteinGym: the *memory* advantage is decisive and directly observable as an OOM boundary crossed by the baseline and not by RMT (Table 4), while the *compute* advantage in raw MACs is not yet visible at these lengths – the linear projection term still dominates the quadratic attention term for both model sizes tested, so RMT’s MACs curve tracks consistently *above* the full-sequence baseline (by the fixed m/b tax) rather than below it, right up to the point where the baseline fails outright. In other words, for this project’s

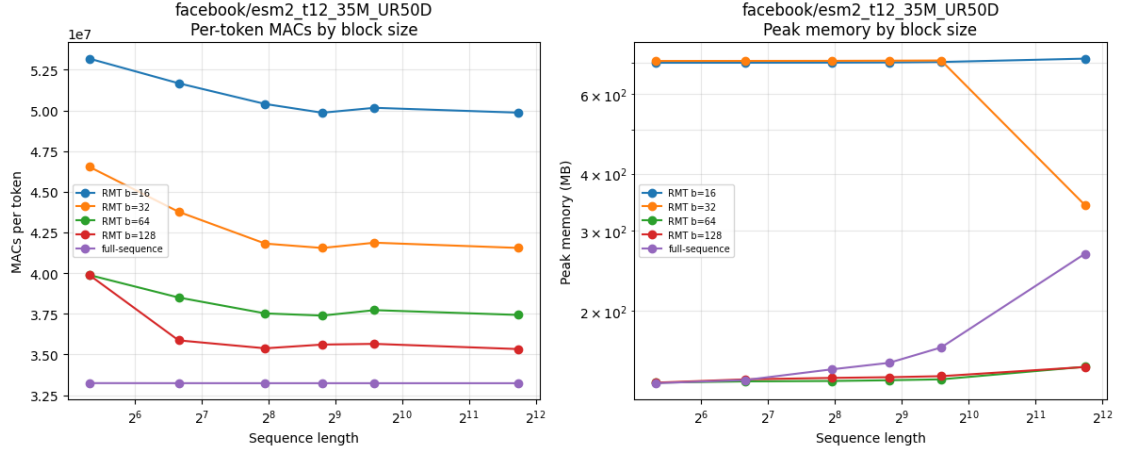


Figure 5: Per-token MACs and peak memory by block size, `esm2_t12_35M_UR50D`.

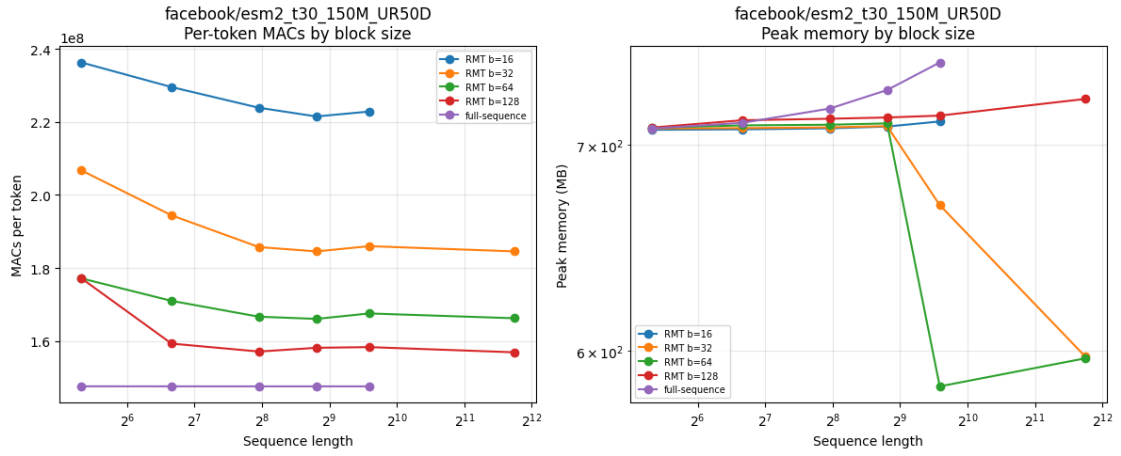


Figure 6: Per-token MACs and peak memory by block size, `esm2_t30_150M_UR50D`.

actual dataset, RMT’s practical value is a memory-scaling story, not (yet) a compute-scaling one – a nuance the asymptotic notation alone does not communicate, but the “ideal vs. effective” framing established in the theoretical section anticipates directly, since it explicitly separates which cost term dominates at a given n rather than asserting linear beats quadratic unconditionally.

The accuracy results (Table 5, Figure 7) quantify the other side of the RMT cons listed in Section 2: the compression bottleneck is real and its cost is a smooth, controllable function of block size, not a step function. This is actionable information rather than a simple pro/con verdict – a practitioner constrained by GPU memory has a continuum of block sizes to choose from, each buying a specific, measured amount of memory savings at a specific, measured accuracy cost, with $b = 128$ recovering near-baseline accuracy on short proteins while $b = 16$ trades far more accuracy for far less headroom.

The ablation results add a mechanistic layer underneath that trade-off curve: they show that the benefit is not merely from adding parameters (memory tokens) to attend over, but specifically from the recurrence connecting them across blocks, and that this recurrence is essentially free in memory terms. This suggests that, of the two “instructions” ablated, recurrence is the effective one and memory-token count is a secondary, more gradual lever – useful information for prioritizing future work, e.g. whether it is more promising to increase m further or to improve how the carried memory state itself is used.

5.1 On biologically-informed block splitting

The theoretical write-up notes that the block-splitting strategy could plausibly be informed by protein structure – motifs, binding epitopes, or structural domains – rather than a fixed arbitrary size, and that testing every possible splitting scheme would sacrifice interpretability of *why* some splittings work better than others. The empirical block-size sweep in this report (fixed $b \in \{128, 64, 32, 16\}$) does not yet test biologically motivated splitting directly; it establishes the cost-accuracy trade-off curve for uniform block sizes first, which is a necessary baseline before a more targeted, structure-aware splitting strategy can be

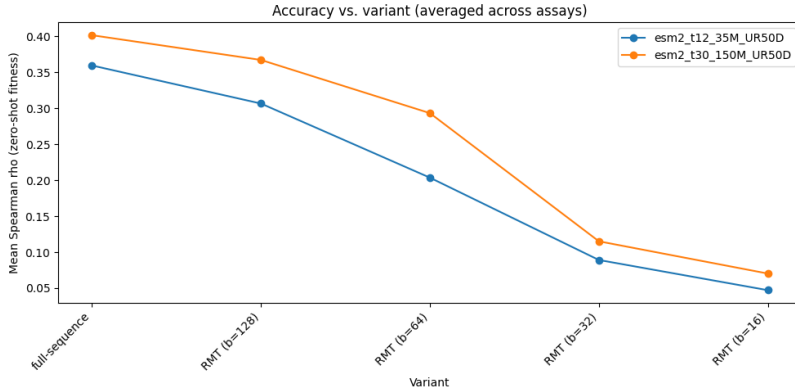


Figure 7: Mean zero-shot fitness-prediction accuracy (Spearman ρ) by variant, averaged across all six assays.

Table 5: Mean Spearman ρ by length bucket, model, and variant

Model	Variant	Short	Medium	Long
esm2_t12_35M_UR50D	full-sequence	0.469	0.453	0.156
	RMT ($b = 128$)	0.478	0.273	0.168
	RMT ($b = 64$)	0.350	0.191	0.069
	RMT ($b = 32$)	0.083	0.110	0.074
	RMT ($b = 16$)	0.040	0.026	0.075
esm2_t30_150M_UR50D	full-sequence	0.531	0.513	0.162
	RMT ($b = 128$)	0.530	0.405	0.167
	RMT ($b = 64$)	0.516	0.292	0.072
	RMT ($b = 32$)	0.137	0.133	0.075
	RMT ($b = 16$)	0.065	0.078	0.068

evaluated against it.

6 Limitations

- **Small assay sample.** Accuracy results are averaged over only six assays (two per length bucket), and the ablation’s accuracy figures over only four. Bucket-level accuracy in particular should be read with this in mind – the long-bucket numbers in Table 5 are heavily influenced by a single difficult assay rather than reflecting a robust trend across long proteins in general.
- **Untrained memory tokens.** The memory-token embeddings are randomly initialized and never fine-tuned on any downstream objective; all accuracy numbers reflect this zero-shot, retrofit setting rather than the upper bound achievable if the memory mechanism were trained end-to-end.
- **One block-splitting strategy.** All block sizes tested use uniform, arbitrary-boundary splitting rather than any biologically motivated segmentation (see Section 5.1).
- **Compute budget.** Block sizes 8 and 3 (motivated by motif-scale biological segmentation) were planned but not included in this round of measurements due to the project’s compute and time budget; one (model, block size) combination (150M, $b = 16$) was excluded entirely due to a runtime-crashing failure mode rather than measured.
- **No wall-clock latency measurement.** Cost is reported in MACs and peak memory only; the sequential-latency cost of RMT’s s segment passes, discussed as a theoretical concern in Section 2, was not directly measured in wall-clock time in this round of experiments.

7 Conclusion

This report derived, and then empirically tested, the cost trade-off between full-sequence attention and RMT-style block-recurrent inference for protein language models. The theoretical analysis predicts $O(n^2)$ time and memory for full-sequence attention against $O(n)$ time and constant memory for RMT. The empirical measurements confirm the memory side of this prediction directly and decisively: on the longest available real protein sequence, the full-sequence baseline exhausts GPU memory while the RMT variant does not, using less memory than the baseline needs on proteins an order of magnitude shorter. The compute side

Table 6: Full ablation results (esm2_t12_35M_UR50D, $b = 32$, cost on $n = 3423$)

m	carry_memory	MACs	Peak mem. (MB)	Mean ρ
0	True	1.146×10^{11}	302.49	0.0890
2	True	1.217×10^{11}	302.97	0.0918
2	False	1.217×10^{11}	302.97	0.0877
4	True	1.288×10^{11}	303.34	0.0924
4	False	1.288×10^{11}	303.34	0.0876
8	True	1.431×10^{11}	304.14	0.1001
8	False	1.431×10^{11}	304.14	0.0938
16	True	1.715×10^{11}	305.74	0.1056
16	False	1.715×10^{11}	305.74	0.0949

of the prediction is confirmed in its qualitative shape (RMT scales more favorably as b shrinks) but is not yet visible as an outright MACs advantage at the sequence lengths naturally available in this dataset, where RMT instead carries a small, consistent, theoretically predicted overhead from its memory-token tax. This advantage is not free: accuracy, measured via zero-shot fitness prediction, degrades smoothly and consistently as block size shrinks, and the targeted ablation shows that this cost is driven specifically by the recurrence mechanism doing genuine work – not merely by the presence of extra tokens – while that same recurrence costs nothing extra in memory. Together, these results characterize RMT not as a strictly better or worse alternative to full-sequence attention, but as a specific, measured point on a memory–accuracy trade-off curve that becomes actionable once the numbers behind it are known.

References

1. Bulatov, A., Kuratov, Y., & Burtsev, M. (2022). Recurrent Memory Transformer. *Advances in Neural Information Processing Systems*.
2. Hutchins, D., Schlag, I., Wu, Y., Dyer, E., & Neyshabur, B. (2022). Block-Recurrent Transformers. *Advances in Neural Information Processing Systems*.
3. Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., & Kaiser, L. (2019). Universal Transformers. *International Conference on Learning Representations*.
4. Lin, Z., Akin, H., Rao, R., et al. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637), 1123–1130. (ESM-2)
5. Notin, P., Kollasch, A., Ritter, D., et al. (2023). ProteinGym: Large-Scale Benchmarks for Protein Fitness Prediction and Design. *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*. <https://proteingym.org/>
6. Meta AI Research. **fvcore**: A light-weight core library for computer vision research, including FLOP/-MAC counting utilities. <https://github.com/facebookresearch/fvcore>